



Logging, informes y automatización

Teoría breve: qué guardar cuando comparas modelos y cómo presentar resultados.

1. Qué registrar por cada run

Campo	Por qué
<code>timestamp</code>	Comparar ejecuciones en el tiempo
<code>pregunta_id</code>	Trazabilidad
<code>modelo</code>	Candidato evaluado
<code>elapsed_ms</code>	Latencia
<code>prompt_tokens</code> / <code>output_tokens</code>	Coste
<code>respuesta</code> (o hash)	Auditoría / revisión humana
<code>temperature</code>	Reproducibilidad
<code>error</code> (si falla)	Robustez operativa

Formato recomendado: **CSV** (fácil de abrir en Excel/pandas) + **report.md** resumen.

2. Estructura de carpetas de salida

```
output/  
├─ benchmark_20250603_143022.csv  
└─ report_20250603_143022.md
```

No versionar `output/` en Git — generado en cada ejecución.

3. Informe Markdown mínimo

Un buen `report.md` incluye:

1. Fecha y modelos evaluados.
 2. Tabla: media de `elapsed_ms` por modelo.
 3. Tabla: media de tokens de salida por modelo.
 4. Enlace o ruta al CSV completo.
 5. **Decisión preliminar** (una frase) o “pendiente revisión humana”.
-

4. Automatización sin magia

```
# Patrón del proyecto ejemplo
from benchmark import ejecutar_benchmark
from report import guardar_csv, generar_reporte_md

filas = ejecutar_benchmark()
csv_path = guardar_csv(filas)
generar_reporte_md(filas, csv_path)
```

El valor está en **congelar** preguntas y parámetros en `config.py` / `data/`.

5. Extensiones del informe (opcionales)

- CI que falla si la latencia p95 sube X %.
- Dataset en JSONL con respuestas esperadas.
- Scoring automático (JSON válido, regex, embedding similarity).

Para este sprint: CSV + revisión manual de una muestra basta.